

VisionInspect AI

Manufacturing Defect Detection & Quality Inspection System

Name: Ruchira Nayak

Internship: Infosys Springboard Virtual Internship

Module: Image Acquisition, Dataset Loading & Image Preprocessing

Milestone: Week 1 (Milestone 1)

Table of Contents

- 1.Project Description
- 2.Project Objective
- 3.My Contribution
- 4.Project Structure
- 5.Dataset Used
- 6.Backend Architecture
- 7.Image Acquisition Module
- 8.Dataset Loading Module
- 9.Image Preprocessing Module
- 10.API Endpoints Developed
- 11.Code Explanation
- 12.Results
- 13.Challenges Faced
- 14.Future Work
- 15.Conclusion

1. Project Description

VisionInspect AI is an AI-powered manufacturing quality inspection platform designed to automate defect detection and quality assessment in industrial products. Traditionally, quality inspection is performed manually by human inspectors, making the process time-consuming, inconsistent, and prone to human error. The goal of this project is to reduce manual effort by integrating Artificial Intelligence, Computer Vision, and Deep Learning into the manufacturing workflow.

The complete system accepts product images captured from production lines or uploaded manually by users. These images are processed using computer vision techniques before being passed to machine learning models for product identification, anomaly detection, defect classification, and quality assessment. Based on the detected defects, the system assigns a severity score and finally generates a pass or fail decision. The platform also provides analytics dashboards that help manufacturers monitor production quality and defect trends over time.

This project serves as a foundation for Industry 4.0 smart manufacturing by enabling automated visual inspection, improving product quality, reducing operational costs, and increasing production efficiency.

2. Project Objective

The primary objective of my assigned module was to build the backend foundation required before training computer vision models. My responsibilities included developing image upload functionality, loading the manufacturing dataset, validating uploaded images, implementing image preprocessing techniques, and generating

processed datasets suitable for future anomaly detection and defect classification models.

The major objectives achieved during this milestone include:

- Building the backend structure using FastAPI.
- Implementing image upload functionality.
- Validating uploaded images before processing.
- Loading the MVTec AD dataset.
- Developing a complete preprocessing pipeline for manufacturing images.
- Processing the entire dataset consisting of 15 industrial object categories.
- Preparing clean images for the upcoming anomaly detection module.

3. My Contribution

During Milestone 1, I was responsible for the Image Acquisition, Dataset Loading, and Image Preprocessing modules of VisionInspect AI.

The following tasks were successfully completed:

- Designed the backend folder structure following modular architecture.
- Developed image upload APIs using FastAPI.
- Implemented image validation for supported file formats.
- Created dataset loading APIs to read the MVTec AD dataset.
- Developed an image preprocessing API for uploaded images.

- Built a complete preprocessing pipeline for the entire dataset.
- Successfully processed **5,354 manufacturing images**.
- Organized processed images into separate output folders.
- Integrated Swagger API documentation for testing backend endpoints.

These modules establish the foundation for future model training, anomaly detection, and defect classification.

4. Project Structure

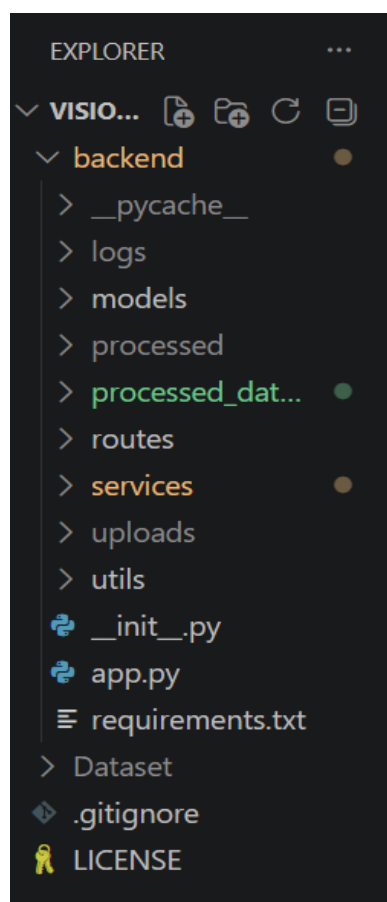
To maintain scalability and modularity, the backend project was organized into multiple components.

```
backend/  
├── routes/  
├── services/  
├── utils/  
├── models/  
├── uploads/  
├── processed/  
├── processed_dataset/  
├── app.py  
└── requirements.txt
```

Each folder has a dedicated responsibility:

- **routes/** – API endpoints.
- **services/** – Business logic and preprocessing.
- **utils/** – Validation functions.
- **models/** – Response models.
- **uploads/** – Uploaded images.
- **processed/** – Preprocessed uploaded images.
- **processed_dataset/** – Complete preprocessed MVTec dataset.

Backend folder structure:



5. Dataset Used

The project uses the **MVTec Anomaly Detection (MVTec AD)** dataset, one of the most widely used benchmark datasets for industrial anomaly detection.

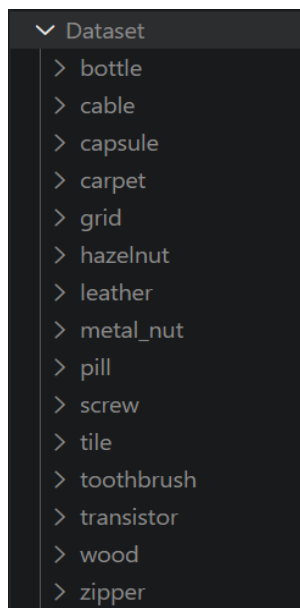
The dataset contains images of real industrial products with both normal and defective samples. It includes pixel-level annotations for defects and supports the development of anomaly detection models.

Dataset Characteristics:

- Total Categories: 15
- Industrial Products: Bottle, Cable, Capsule, Carpet, Grid, Hazelnut, Leather, Metal Nut, Pill, Screw, Tile, Toothbrush, Transistor, Wood, and Zipper.
- Training Images: Normal (Good) images.
- Testing Images: Both normal and defective images.
- Ground Truth: Pixel-level defect masks.
- Total Images Processed: **5,354**

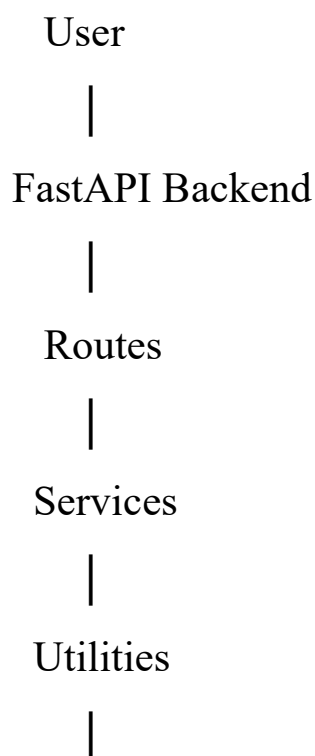
The dataset was organized into category-wise folders, each containing separate training and testing directories.

Dataset folder:



6. Backend Architecture

The backend was developed using FastAPI following a layered architecture.



Processed Images

The workflow begins when the user uploads an image. The image passes through validation and preprocessing before being saved. Dataset loading services provide category information, while preprocessing services prepare the images for future anomaly detection models.

7. Image Acquisition Module

The Image Acquisition module enables users to upload manufacturing images through the backend API.

The upload process performs the following operations:

1. Accepts the uploaded image.
2. Validates the file extension.
3. Verifies that the uploaded file is a valid image.
4. Saves the image to the uploads directory.
5. Returns a success response containing the filename and storage path.

Supported formats include:

- JPG
- JPEG
- PNG
- BMP
- TIFF
- WEBP

The module ensures that invalid or corrupted files cannot enter the processing pipeline.

Swagger /upload API (before execution)

The image shows the Swagger UI for the `POST /upload` endpoint. The interface is divided into several sections:

- Parameters:** No parameters are defined for this endpoint.
- Request body:** Labeled as *required*, with a dropdown menu set to `multipart/form-data`. A field `file` is listed as *required* with a type of `string`.
- Responses:** A table showing the response for a `200` status code.

Code	Details
200	Response body <pre>{ "message": "Image uploaded successfully", "filename": "005.png", "saved_path": "backend/uploads\\005.png"}</pre> Response headers <pre>content-length: 102 content-type: application/json date: Thu, 09 Jul 2026 05:59:15 GMT server: uvicorn</pre>

Below the response table, there is a **Curly** section with a curl command and a **Request URL** section with the URL `http://127.0.0.1:8000/upload`.

Swagger /upload successful response:

This image shows the Swagger UI for the `POST /upload` endpoint, displaying the successful response details. The interface includes a sidebar with navigation icons and a top bar with file names like `dataset_preprocessor.py`, `upload.py`, and `image_processor.py`.

The **Responses** section is expanded, showing the details for the `200` status code:

- Response body:** A JSON object with the following structure:

```
{  "message": "Image uploaded successfully",  "filename": "005.png",  "saved_path": "backend/uploads\\005.png"}
```
- Response headers:** A list of headers:

```
content-length: 102  content-type: application/json  date: Thu, 09 Jul 2026 05:53:40 GMT  server: uvicorn
```

Below the response details, there is a **Responses** table with columns **Code**, **Description**, and **Links**.

Code	Description	Links
200	Successful Response	No links

Under the **Successful Response** row, there is a **Media type** dropdown set to `application/json`, a **Controls** section with **Accept header** and **Example Value** (set to `"string"`), and a **Schema** section.

8. Dataset Loading Module

The Dataset Loading module scans the MVTec AD dataset and identifies all available product categories.

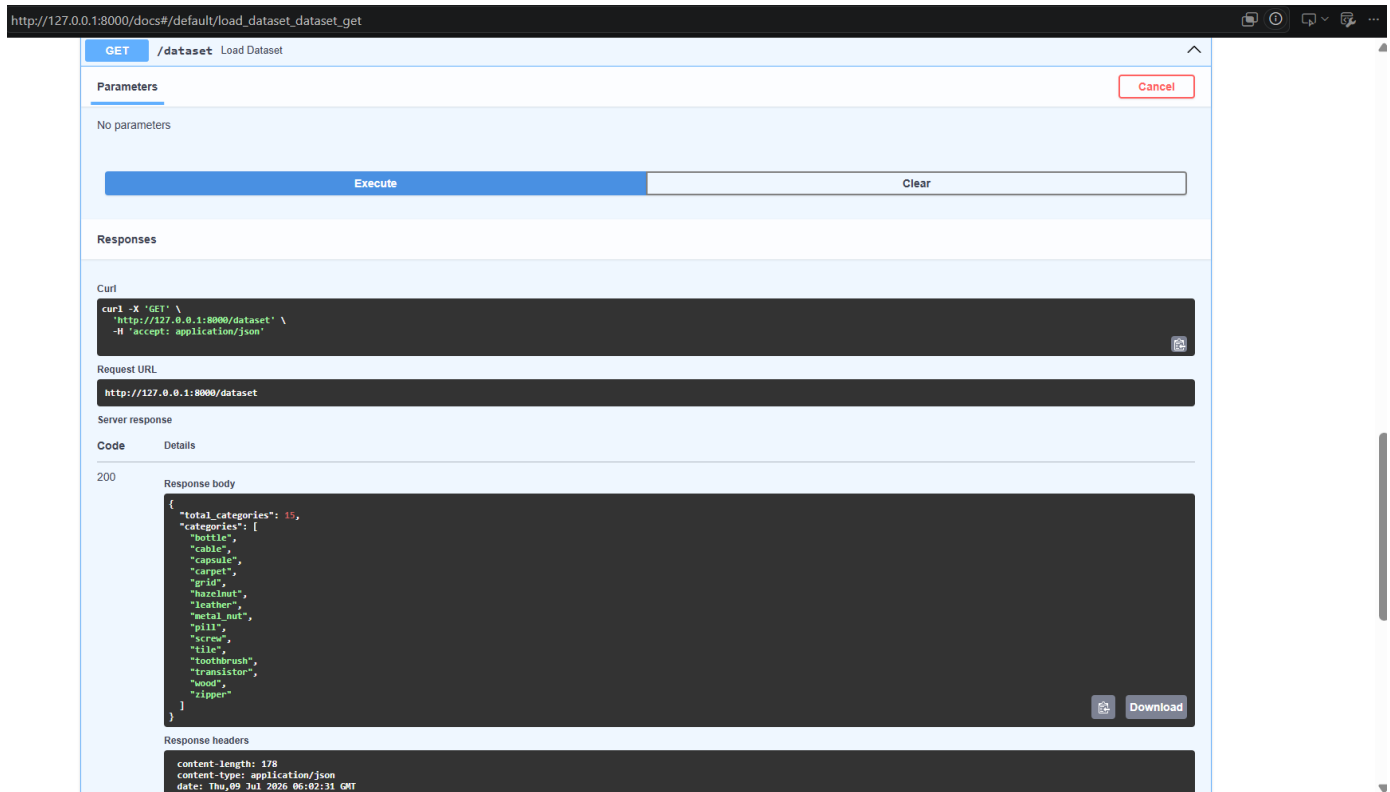
Instead of manually specifying category names, the API dynamically reads the dataset directory and returns all available classes.

This provides flexibility, allowing new categories to be added without modifying the source code.

The API successfully identified the following categories:

- Bottle
- Cable
- Capsule
- Carpet
- Grid
- Hazelnut
- Leather
- Metal Nut
- Pill
- Screw
- Tile
- Toothbrush
- Transistor
- Wood
- Zipper

/dataset response showing 15 categories:



9. Image Preprocessing Module

Image preprocessing plays a critical role in improving image quality before machine learning models perform defect detection.

A dedicated preprocessing pipeline was implemented to prepare both uploaded images and the complete MVTec dataset.

The preprocessing pipeline consists of the following stages:

Step 1 – RGB Conversion

Images loaded using OpenCV are converted from BGR format to RGB format to maintain color consistency.

Step 2 – Noise Removal

Gaussian Blur is applied to remove small sensor noise and improve image smoothness.

Step 3 – Contrast Enhancement

Contrast Limited Adaptive Histogram Equalization (CLAHE) enhances image contrast, making manufacturing defects more visible.

Step 4 – Image Sharpening

A sharpening kernel is applied to highlight edges and improve visibility of scratches, cracks, and surface defects.

After implementing the preprocessing algorithm, a dataset preprocessing script was developed to automatically process every image across all categories.

The script traversed the complete dataset structure and successfully processed **5,354 images**, storing them inside a separate processed dataset directory.

dataset_preprocessor.py (imports + dataset path):

```
backend > services > dataset_preprocessor.py > preprocess_image
1  import os
2  import cv2
3  import numpy as np
4
5  # Dataset location
6  DATASET_PATH = "Dataset"
7
8  # Output folder
9  OUTPUT_PATH = "backend/processed_dataset"
10 os.makedirs(OUTPUT_PATH, exist_ok=True)
```

preprocess_image() function (RGB → Blur → CLAHE → Sharpen)

```
backend > services > dataset_preprocessor.py > preprocess_image
11
12 def preprocess_image(image):
13     """
14     Preprocess one image.
15     """
16
17     # Convert BGR to RGB
18     image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
19
20     # Remove small noise
21     image = cv2.GaussianBlur(image, (3, 3), 0)
22
23     # Convert to LAB color space
24     lab = cv2.cvtColor(image, cv2.COLOR_RGB2LAB)
25
26     l, a, b = cv2.split(lab)
27
28     # Contrast enhancement
29     clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8,8))
30     l = clahe.apply(l)
31
32     lab = cv2.merge((l, a, b))
33
34     image = cv2.cvtColor(lab, cv2.COLOR_LAB2RGB)
35
36     # Sharpen
37     kernel = np.array([
38         [0,-1,0],
39         [-1,5,-1],
40         [0,-1,0]
41     ])
42
43     image = cv2.filter2D(image, -1, kernel)
44
45     return image
```

Terminal output showing "Dataset Preprocessing Completed Successfully – Total Images Processed: 5354:

```
Processing Category: pill
Processed 434 images.

Processing Category: screw
Processed 480 images.

Processing Category: tile
Processed 347 images.

Processing Category: toothbrush
Processed 102 images.

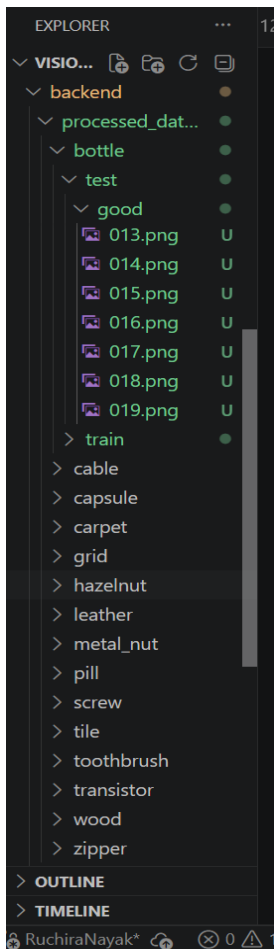
Processing Category: transistor
Processed 313 images.

Processing Category: wood
Processed 326 images.

Processing Category: zipper
Processed 391 images.

=====
Dataset Preprocessing Completed Successfully!
Total Images Processed : 5354
=====
```

backend/processed_dataset folder:

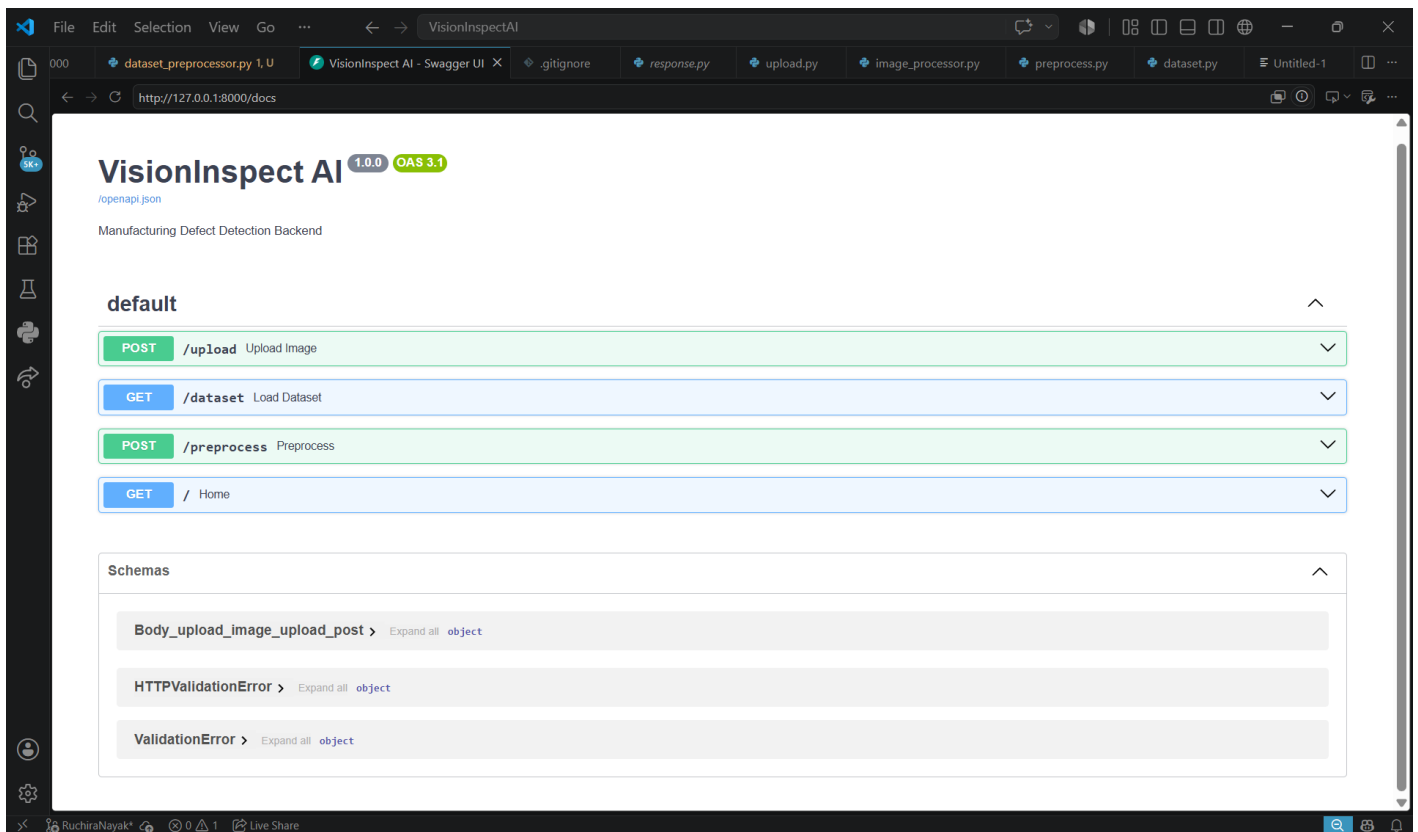


10. API Endpoints Developed

Endpoint	Method	Description
/	GET	Backend home endpoint
/upload	POST	Upload manufacturing image
/dataset	GET	Load available dataset categories
/preprocess	POST	Preprocess uploaded image

These APIs were tested using FastAPI Swagger UI.

Swagger homepage showing all APIs:



11. Code Explanation

The backend implementation follows a modular architecture to improve readability and maintainability.

app.py

Initializes the FastAPI application and registers all API routes.

upload.py

Handles image uploads, validation, and file storage.

validation.py

Verifies supported image extensions and validates uploaded image files.

dataset_loader.py

Scans the dataset directory and returns available manufacturing object categories.

image_processor.py

Implements preprocessing for individual uploaded images.

dataset_preprocessor.py

Processes the complete MVTec AD dataset by applying:

- RGB Conversion
- Gaussian Blur
- CLAHE Contrast Enhancement
- Image Sharpening

The processed images are stored separately for future model training.

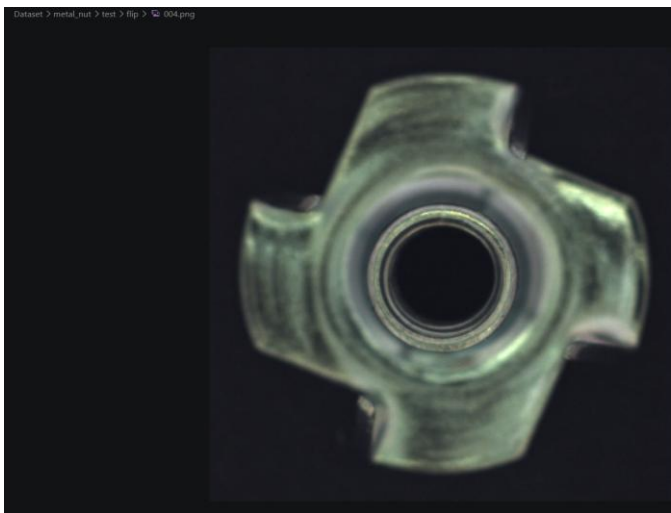
12. Results

The developed modules were successfully tested.

Results obtained:

- Backend APIs functioning correctly.
- Image upload working successfully.
- Dataset loading completed successfully.
- Image preprocessing API functioning correctly.
- Complete MVTec dataset processed successfully.
- Total Images Processed: **5,354**
- Output Directory Generated:
backend/processed_dataset

The processed dataset is now ready for anomaly detection model training in the upcoming milestone.



Original image



Preprocessed image

13. Challenges Faced

During implementation, several technical challenges were encountered:

- Configuring the FastAPI backend structure.
- Resolving Python package installation and PATH issues.
- Fixing import path errors during backend execution.
- Handling the large MVTec AD dataset efficiently.
- Managing GitHub repository limitations for a 4.9 GB dataset.
- Designing a preprocessing pipeline suitable for all industrial object categories.

Each issue was resolved through debugging, testing, and iterative improvements.

14. Future Work

The completed backend modules provide the foundation for future milestones.

Upcoming work includes:

- Integrating YOLO for product detection.
- Training anomaly detection models.
- Implementing defect localization.
- Developing defect classification.
- Calculating severity scores.
- Building quality control reports.
- Integrating manufacturing analytics dashboards.

- Deploying the complete platform using Docker and cloud services.

15. Conclusion

During Milestone 1, the backend foundation of VisionInspect AI was successfully established. Image acquisition, dataset loading, and image preprocessing modules were developed and tested using FastAPI. The MVTec AD dataset was successfully integrated, and a preprocessing pipeline was implemented to enhance manufacturing images through RGB conversion, noise removal, contrast enhancement, and sharpening. A total of **5,354 images** were processed and organized for future model training.

The completed work provides a reliable and scalable backend infrastructure that supports the upcoming development of anomaly detection, defect classification, and quality inspection modules. This milestone establishes a strong foundation for building an end-to-end AI-powered manufacturing defect detection system.